

DAY 4 - PostgreSQL 18 Backup and Disaster Recovery

Revised lab manual for Zalando Postgres Operator / Patroni: **postgres-ha / acid-postgres-ha**. Custom Docker Hub image: **[docker.io/eswaribala/ericpostgrerepo2026:pg18](https://hub.docker.io/eswaribala/ericpostgrerepo2026:pg18)**. Run commands in Bash. Keep each code block together.

Safety: Backups read from the live cluster. Physical restore and PITR must run only on a separate recovery PVC with PostgreSQL stopped. Never restore over the three active cluster PVCs. This revision replaces the registry.example.com placeholder, fixes validation that included pooler pods, adds private image pull setup, and checks all three members before pgBackRest stanza creation.

0. Discover your live cluster

Contents: 0 Cluster checks; 1 Logical backup; 2 pg_basebackup; 3 WAL archiving; 4 Immutable image and rollout; 5 GCS repository; 6 Stanza and WAL migration; 7 Full, incremental and differential backups; 8 Isolated restore and PITR; 9 Troubleshooting; 10 Failover, retention, cleanup and lab checklist.

Architecture: Applications reach the acid-postgres-ha primary Service. Patroni manages the three PostgreSQL pods. Logical and physical backups read the running database. WAL segments and pgBackRest backups go to a separate durable repository. Recovery reads that repository into a distinct recovery PVC. PgBouncer pooler pods serve connections but do not store PostgreSQL data or need pgBackRest.

Concepts: pg_dump stores SQL-level data, pg_basebackup creates a physical base copy, and pgBackRest manages full and changed-block physical backups plus archived WAL. A full backup is a base; a differential depends on the most recent full; an incremental depends on the preceding backup. PITR combines a physical backup with archived WAL to reach a chosen point. Patroni failover keeps service available, but does not undo accidental deletes propagated to replicas.

Prerequisites: PostgreSQL 18 cluster healthy; kubectl access to the namespace; Bash, jq, docker and PostgreSQL client tools on the workstation; enough disk for dumps and basebackup; an existing GCS bucket with authorized workload identity; Docker Hub access to eswaribala/ericpostgrerepo2026; and an isolated PVC for restore. Confirm your operator version supports the CR fields before editing.

```
NS=postgres-ha
CLUSTER=acid-postgres-ha
CUSTOM_IMAGE=docker.io/eswaribala/ericpostgrerepo2026:pg18
kubectl -n "$NS" get postgresql "$CLUSTER"
kubectl -n "$NS" get pods -o wide
kubectl -n "$NS" get svc,pvc
PRIMARY=''
for p in "$CLUSTER"-0 "$CLUSTER"-1 "$CLUSTER"-2; do
  role=$(kubectl -n "$NS" exec "$p" -c postgres -- \
    curl -fsS http://127.0.0.1:8008/ | jq -r '.role')
  echo "$p $role"
  case "$role" in primary|master|leader) PRIMARY="$p";; esac
done
test -n "$PRIMARY" && echo "Primary=$PRIMARY"
```

Exactly one member should be primary. Recheck PRIMARY after failover. PgBouncer poolers can share cluster-name labels, so the ordinal pod names above deliberately select only PostgreSQL members.

1. Lab 1 - Logical backup

```
mkdir -p backups/logical
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  pg_dumpall -U postgres > backups/logical/cluster.sql
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  pg_dump -U postgres -Fc banking > backups/logical/banking.dump
pg_restore -l backups/logical/banking.dump | head -30
ls -lh backups/logical/
```

Test a logical restore into a separate test database or recovery cluster. The pg_dump files are not pgBackRest backup sets.

1.1 What to inspect: The SQL file should be nonempty. A custom-format dump should list schemas, tables, data and indexes with pg_restore -l. A pg_dump of banking does not include unrelated cluster roles; pg_dumpall includes global objects. Keep the backups in durable storage after this local demonstration.

```
head -25 backups/logical/cluster.sql
pg_restore -l backups/logical/banking.dump | head -40
```

1.2 Optional logical restore test: Only in a disposable lab database on the live cluster, create a separate banking_restore_test database and pipe the custom dump into pg_restore. Do not run --clean on banking itself.

```
kubect1 -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -c 'CREATE DATABASE banking_restore_test;\'
cat backups/logical/banking.dump | kubect1 -n "$NS" exec -i \
  "$PRIMARY" -c postgres -- pg_restore -U postgres \
  -d banking_restore_test --no-owner
kubect1 -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -d banking_restore_test -c '\dt'
```

If the test database already exists, choose another name. Object ownership or installed extensions may require matching roles and extensions. Do not delete the test database until the restore result has been verified.

2. Lab 2 - Physical backup with pg_basebackup

Use the operator standby secret and the primary Service; 127.0.0.1 inside a replica pod addresses that replica. Confirm the secret name with kubect1 get secrets.

```
PRIMARY_SERVICE=acid-postgres-ha
REPLICA="$CLUSTER-1"
test "$REPLICA" != "$PRIMARY" || REPLICA="$CLUSTER-2"
kubect1 -n "$NS" get svc "$PRIMARY_SERVICE"
kubect1 -n "$NS" get secrets | grep -Ei 'standby|replic'
REPL_SECRET=standby.acid-postgres-ha.credentials.postgresql.acid.zalan.do
REPL_USER=$(kubect1 -n "$NS" get secret "$REPL_SECRET" \
  -o jsonpath='{.data.username}' | base64 -d)
REPL_PASS=$(kubect1 -n "$NS" get secret "$REPL_SECRET" \
  -o jsonpath='{.data.password}' | base64 -d)
kubect1 -n "$NS" exec "$REPLICA" -c postgres -- \
  env PGPASSWORD="$REPL_PASS" pg_basebackup \
  -h "$PRIMARY_SERVICE" -p 5432 -U "$REPL_USER" \
  -D /tmp/basebackup -Ft -z -Xs -P
mkdir -p backups/physical
kubect1 -n "$NS" cp -c postgres \
  "$REPLICA:/tmp/basebackup" backups/physical/basebackup
unset REPL_PASS
```

Check free space in /tmp first. A backup left inside a pod can disappear on restart. If pg_hba rejects the connection, check replication rules and SSL instead of adding trust rules.

2.1 Precheck: Query the primary for the replication role, HBA entries and connected standbys. A standby credential can exist yet still be denied by source address, SSL policy or pg_hba ordering.

```
kubect1 -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -c "SELECT rolname,rolreplication,rolcanlogin
  FROM pg_roles WHERE rolreplication;"
kubect1 -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -c "SELECT line_number,type,database,user_name,
  address,auth_method,error FROM pg_hba_file_rules
  WHERE 'replication'=ANY(database);"
kubect1 -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -c "SELECT application_name,state,sync_state,
  sent_lsn,replay_lsn FROM pg_stat_replication;"
kubect1 -n "$NS" exec "$REPLICA" -c postgres -- df -h /tmp
```

2.2 Validation: The tar format usually creates base.tar.gz and WAL tar output when -Xs is used. The -P option displays transfer progress; -Ft selects tar format; -z compresses; -Xs streams WAL. Confirm the files were copied off the pod.

```
kubectl -n "$NS" exec "$REPLICA" -c postgres -- \
  ls -lh /tmp/basebackup
find backups/physical/basebackup -maxdepth 2 -type f -ls
```

3. Lab 3 - Inspect existing WAL archiving

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -c 'SHOW archive_mode;'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -c 'SHOW archive_command;'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -x -c 'SELECT * FROM pg_stat_archiver;'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -c \
  'command -v wal-g || true; command -v wal-e || true'
```

Gate: If WAL-G/WAL-E currently handles archiving, plan continuity before switching archive_command. Preserve its old repository until the first pgBackRest full backup and its WAL chain are proven recoverable.

3.1 Force one WAL switch: Record archived_count and failed_count, switch WAL on the actual primary, then inspect the counters and last archived file. Archiving can take longer than ten seconds; wait and recheck rather than interpreting a single delayed result as failure.

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -c "SELECT archived_count,failed_count,
  last_archived_wal,last_archived_time FROM pg_stat_archiver;"
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -c 'SELECT pg_switch_wal();'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -c "SELECT archived_count,failed_count,
  last_archived_wal,last_archived_time,last_failed_wal
  FROM pg_stat_archiver;"
```

Expected: archived_count advances eventually without a new failed_count. PostgreSQL may use archive_library instead of archive_command; inspect both settings. A healthy Patroni replica is not itself an off-cluster backup.

4. Lab 4 - Configure pgBackRest: image and rollout

4.1 Save the current image and operator resources. These YAMLS are snapshots for review; do not apply the StatefulSet snapshot over operator-managed resources.

```
CURRENT_IMAGE=$(kubectl -n "$NS" get pod "$PRIMARY" \
  -o jsonpath='{.spec.containers[?(@.name=="postgres")].image}')
echo "$CURRENT_IMAGE"
kubectl -n "$NS" get postgresql "$CLUSTER" -o yaml \
  > "${CLUSTER}-before-pgbackrest.yaml"
kubectl -n "$NS" get statefulset "$CLUSTER" -o yaml \
  > "${CLUSTER}-sts-before-pgbackrest.yaml"
```

4.2 Your image has reportedly already been built and pushed. Verify its tag; if rebuilding, use these commands. Docker Hub stores the **container image**; the GCS bucket below stores **backups**.

Immutable image: The Dockerfile installs pgBackRest during docker build. Packages installed manually with kubectl exec are lost when a pod is replaced. The tag pg18 is convenient for training, but use a versioned tag or pinned digest for repeatable production rollouts. The original registry.example.com name is only an example and must not appear in the new CR.

```
cat > Dockerfile.spilo-pgbackrest <<'EOF'
ARG BASE_IMAGE
FROM ${BASE_IMAGE}
```

```

USER root
RUN apt-get update && apt-get install -y --no-install-recommends pgbackrest \
    && rm -rf /var/lib/apt/lists/*
USER postgres
EOF
docker build --build-arg BASE_IMAGE="$CURRENT_IMAGE" \
    -f Dockerfile.spilo-pgbackrest -t "$CUSTOM_IMAGE" .
docker run --rm --entrypoint pgbackrest "$CUSTOM_IMAGE" version
docker login
docker push "$CUSTOM_IMAGE"
docker image inspect "$CUSTOM_IMAGE" --format '{{json .RepoDigests}}'

```

4.3 Your Docker Hub repository is private. Create a namespace pull secret using a Docker Hub access token. The token is read interactively; do not paste it into the manual or commit it. **Creating a secret alone is insufficient:** the generated StatefulSet pod template must reference it. Configure imagePullSecrets using a supported field in your installed operator version, then inspect the template. Alternatively, make only this training image public if appropriate.

You can inspect the PostgreSQL pod service account and its imagePullSecrets. An operator may set serviceAccountName itself, so do not assume patching the default service account will affect these pods.

```

kubectl -n "$NS" get statefulset "$CLUSTER" \
    -o jsonpath='{.spec.template.spec.serviceAccountName}'
kubectl -n "$NS" get pod "$PRIMARY" \
    -o jsonpath='{.spec.imagePullSecrets}'

read -rsp 'Docker Hub access token: ' DOCKERHUB_TOKEN; echo
kubectl -n "$NS" create secret docker-registry dockerhub-secret \
    --docker-server=https://index.docker.io/v1/ \
    --docker-username=eswaribala \
    --docker-password="$DOCKERHUB_TOKEN" \
    --dry-run=client -o yaml | kubectl -n "$NS" apply -f -
unset DOCKERHUB_TOKEN
kubectl -n "$NS" get statefulset "$CLUSTER" \
    -o jsonpath='{.spec.template.spec.imagePullSecrets}'

```

4.4 Edit the PostgreSQL custom resource and set **spec.dockerImage: docker.io/eswaribala/ericpostgrrepo2026:pg18**. Do this during a maintenance window; it triggers an operator-managed rollout. Verify the generated StatefulSet before expecting pods to change.

```

kubectl -n "$NS" edit postgresql "$CLUSTER"
kubectl -n "$NS" get postgresql "$CLUSTER" \
    -o jsonpath='{.spec.dockerImage}'
kubectl -n "$NS" get statefulset "$CLUSTER" \
    -o jsonpath='{.spec.template.spec.containers[?(@.name=="postgres")].image}'
kubectl -n "$NS" rollout status statefulset/"$CLUSTER" --timeout=15m

```

If the StatefulSet image is old, inspect operator reconciliation. If ImagePullBackOff appears, run kubectl describe pod to inspect events and fix the registry pull secret. Do not delete a healthy pod to force a rollout before the pod template is correct.

4.5 Validate the three database pods, excluding pooler pods. All three must have the custom image and pgBackRest. Check imageID to catch a reused mutable tag.

```

for p in "$CLUSTER"-0 "$CLUSTER"-1 "$CLUSTER"-2; do
    echo "=== $p ==="
    kubectl -n "$NS" get pod "$p" \
        -o jsonpath='{.spec.containers[?(@.name=="postgres")].image}'
    kubectl -n "$NS" get pod "$p" \
        -o jsonpath='{.status.containerStatuses[?(@.name=="postgres")].imageID}'
    kubectl -n "$NS" exec "$p" -c postgres -- pgbackrest version
done

```

Stop gate: The earlier output showed pgBackRest on pod 0 but not pods 1 and 2. Do not run stanza-create until all three PostgreSQL pods pass. Pooler pods need no pgBackRest.

5. Configure the pgBackRest repository

5.1 Choose an existing GCS bucket and ensure the Kubernetes workload identity used by PostgreSQL has access to it. GCS credentials are separate from Docker Hub pull credentials. On a non-GCP Kubernetes cluster, configure a supported GCS credential mechanism; `repo1-gcs-key-type=auto` alone may not work.

```
GCS_BUCKET=REPLACE_WITH_YOUR_GCS_BUCKET
[ "$GCS_BUCKET" != REPLACE_WITH_YOUR_GCS_BUCKET ] || \
{ echo 'Set a real GCS bucket'; false; }
kubectl -n "$NS" get pod "$PRIMARY" \
-o jsonpath='{.spec.serviceAccountName}{"\n"}'
PGDATA_ACTUAL=$(kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
psql -U postgres -Atqc 'SHOW data_directory;')
echo "$PGDATA_ACTUAL"
```

5.2 Make the repository encryption key. Preserve an authorized secure copy outside the cluster: without it, encrypted backups cannot be recovered. The stanza name `postgres-ha` identifies the database cluster.

```
PGBR_CIPHER_PASS=$(openssl rand -base64 48)
kubectl -n "$NS" create secret generic pgbackrest-repo-secret \
--from-literal=repo-cipher-pass="$PGBR_CIPHER_PASS"
unset PGBR_CIPHER_PASS
cat > pgbackrest.conf <<EOF
[postgres-ha]
pg1-path=$PGDATA_ACTUAL
pg1-port=5432

[global]
repo1-type=gcs
repo1-gcs-bucket=$GCS_BUCKET
repo1-gcs-key-type=auto
repo1-path=/pgbackrest/postgres-ha
repo1-retention-full=2
repo1-retention-diff=4
repo1-cipher-type=aes-256-cbc
compress-type=zst
compress-level=3
process-max=2
start-fast=y
log-level-console=info
EOF
kubectl -n "$NS" create configmap pgbackrest-config \
--from-file=pgbackrest.conf=./pgbackrest.conf
```

5.3 Mount `pgbackrest-config` at `/etc/pgbackrest` and `pgbackrest-repo-secret` at `/etc/pgbackrest-secret` **inside the postgres container on all three pods**. Operator CRD fields vary; inspect your installed CRD and use its supported volume/pod-template mechanism. Do not directly patch the generated `StatefulSet`; reconciliation may undo it. The earlier manual lacked a verified universal mount patch.

Recent Zalando operator releases document `spec.additionalVolumes` with `name`, `mountPath` and `volumeSource`. If `kubectl explain postgresql.spec.additionalVolumes` confirms these fields on your cluster, add the following entries under `spec` in the PostgreSQL CR. Preserve all existing `spec` fields. The empty `targetContainers` default mounts in `postgres` only. Validate the generated `StatefulSet` after editing.

```
additionalVolumes:
- name: pgbackrest-config
  mountPath: /etc/pgbackrest
  volumeSource:
    configMap:
      name: pgbackrest-config
- name: pgbackrest-repo-secret
  mountPath: /etc/pgbackrest-secret
  volumeSource:
    secret:
      secretName: pgbackrest-repo-secret
```

The YAML above is a spec fragment, not a complete manifest or shell command. Inspect the existing additionalVolumes list and append instead of replacing other mounts. For a private image, imagePullSecrets still needs an operator-supported configuration path and must appear in the pod template.

```
kubectl explain postgresql.spec --recursive | \
  grep -Ei 'volume|mount|secret|podTemplate|dockerImage' | head -50
kubectl -n "$NS" get statefulset "$CLUSTER" \
  -o jsonpath='{.spec.template.spec.volumes}{"\n\n"}'
for p in "$CLUSTER"-0 "$CLUSTER"-1 "$CLUSTER"-2; do
  kubectl -n "$NS" exec "$p" -c postgres -- \
    test -r /etc/pgbackrest/pgbackrest.conf || break
  kubectl -n "$NS" exec "$p" -c postgres -- \
    test -r /etc/pgbackrest-secret/repo-cipher-pass || break
  echo "$p config-and-secret-ok"
done
```

Stop gate: A ConfigMap or Secret in the namespace does not imply it is mounted. Continue only after every pod can read both files.

5.4 Verify the repository setting: The pg1-path is read from SHOW data_directory, avoiding a guessed Spilo path. repo1-type=gcs selects Cloud Storage; repo1-path is a prefix in the bucket; repo1-cipher-type and the mounted passphrase encrypt backup repository contents; compression reduces transfer and storage; process-max controls parallel workers. Confirm pgBackRest can parse the configuration on each pod before changing archiving.

```
for p in "$CLUSTER"-0 "$CLUSTER"-1 "$CLUSTER"-2; do
  echo "=== $p ==="
  kubectl -n "$NS" exec "$p" -c postgres -- sh -lc \
    'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; \
    pgbackrest --stanza=postgres-ha info'
done
```

Before the first stanza-create, info may report that the stanza is missing; that is expected. Authentication, malformed configuration and missing paths are not expected. Never print the secret value in lab output.

6. Initialize stanza and migrate WAL archiving

Run stanza-create once from the current primary. Then configure archive_command through Patroni/operator-managed configuration. Do not hand-edit postgresql.conf, and do not change from a functioning WAL-G archive without a migration plan. The command below is the command value to use in the Patroni archive_command setting after testing and adapting shell quoting for your operator.

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc \
  'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; \
  pgbackrest --stanza=postgres-ha stanza-create'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- patronictl show-config

PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass) pgbackrest --stanza=postgres-ha archive-push %p
```

In patronictl edit-config, place archive_command under postgresql.parameters. Example configuration fragment (do not replace unrelated Patroni settings):

```
postgresql:
  parameters:
    archive_mode: 'on'
    archive_command: >-
      PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)
      pgbackrest --stanza=postgres-ha archive-push %p
```

Before saving, make sure the command is available to PostgreSQL as its runtime user on every member and the repository identity works after failover. Refresh the actual current primary after the rolling restart. If your operator owns Patroni settings in the PostgreSQL CR, configure them there instead of a transient change that reconciliation overwrites.

```
kubectl -n "$NS" exec -it "$PRIMARY" -c postgres -- \
    patronictl edit-config
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
    patronictl show-config
```

Set `archive_mode=on` if needed; this may require a restart. After operator reconciliation, validate the effective PostgreSQL settings and confirm `archived_count` advances without a new `failed_count`. `pgBackRest` check can rely on functioning WAL archiving, so run it after the switch.

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
    psql -U postgres -c 'SHOW archive_command;'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
    psql -U postgres -c 'SELECT pg_switch_wal();'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc \
    'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; \
    pgbackrest --stanza=postgres-ha check'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
    psql -U postgres -x -c 'SELECT * FROM pg_stat_archiver;'
```

7. Labs 5-7 - Full, incremental, differential backups

Refresh the primary after failover. A full backup anchors the chain; an incremental uses the previous backup; a differential uses the latest full. The labels in `pgBackRest` info usually end F, I, and D respectively.

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc \
    'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; \
    pgbackrest --stanza=postgres-ha --type=full backup; \
    pgbackrest --stanza=postgres-ha info'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc \
    'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; \
    pgbackrest --stanza=postgres-ha --type=incr backup; \
    pgbackrest --stanza=postgres-ha info'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc \
    'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; \
    pgbackrest --stanza=postgres-ha --type=diff backup; \
    pgbackrest --stanza=postgres-ha info'
```

Create approved test changes between backup types to demonstrate size differences. Retention settings are two fulls and four differentials; review the resulting backup dependency chain before running `expire`.

7.1 Demonstrate changes: On a disposable banking training database, create a marker table after the full backup, insert one row before the incremental, then another before the differential. The three backups will then represent different database states.

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
    psql -U postgres -d banking -c "CREATE TABLE IF NOT EXISTS
    dr_lab_events(id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    event_time timestamptz DEFAULT clock_timestamp(), note text);"
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
    psql -U postgres -d banking -c "INSERT INTO dr_lab_events(note)
    VALUES ('before incremental');"
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
    psql -U postgres -d banking -c "INSERT INTO dr_lab_events(note)
    VALUES ('before differential');"
```

Run the inserts at the matching points, not all together after the backups. Record the backup labels, timestamps, backup sizes and WAL ranges from `pgBackRest` info. A differential will usually depend on only a full while a later incremental may depend on multiple intermediate backups.

8. Labs 8-9 - Restore and PITR on isolated recovery storage

Create a **separate recovery PVC and toolbox pod** with the same image, GCS access, ConfigMap and encryption Secret. Stop PostgreSQL on that isolated target. Confirm its PGDATA is an empty writable directory on the recovery PVC. The earlier example that first started an operator-managed recovery cluster and later overwrote its PVC is omitted: bootstrap and restore could conflict. Integrating a restored volume into a new operator cluster requires a version-specific recovery procedure and validation.

8.1 Capture the live pod identity and prepare a distinct PVC: Set REC_SIZE at least as large as the physical database plus WAL and working space. The recovery PVC name must be different from every active acid-postgres-ha PVC. Choose a storage class that can provision the PVC in this namespace.

```
REC_PVC=pgbackrest-recovery-data
REC_SIZE=30Gi
POD_SA=$(kubectl -n "$NS" get pod "$PRIMARY" \
-o jsonpath='{.spec.serviceAccountName}')
PG_UID=$(kubectl -n "$NS" exec "$PRIMARY" -c postgres -- id -u)
PG_GID=$(kubectl -n "$NS" exec "$PRIMARY" -c postgres -- id -g)
echo "$POD_SA $PG_UID $PG_GID"
kubectl -n "$NS" get pvc | grep -E 'acid-postgres-ha|recovery'
```

Stop if REC_PVC equals an existing active cluster PVC. The sample uses ReadWriteOnce; select an appropriate storage class for your cluster. The test pod must schedule on a node that can mount the PVC.

```
cat > recovery-pvc.yaml <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: $REC_PVC
  namespace: $NS
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: $REC_SIZE
EOF
kubectl apply -f recovery-pvc.yaml
kubectl -n "$NS" get pvc "$REC_PVC"
```

8.2 Create the recovery toolbox: It mounts only the newly named PVC. A private Docker Hub image requires the same working pull secret. Its Kubernetes service account also needs the repository IAM rights. Set runAsUser/runAsGroup to the discovered image user, and review your storage class fsGroup behavior if it reports permission denied.

```
cat > recovery-toolbox.yaml <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: pgbackrest-recovery-toolbox
  namespace: $NS
spec:
  restartPolicy: Never
  serviceAccountName: $POD_SA
  imagePullSecrets:
    - name: dockerhub-secret
  securityContext:
    runAsUser: $PG_UID
    runAsGroup: $PG_GID
    fsGroup: $PG_GID
  containers:
    - name: toolbox
      image: $CUSTOM_IMAGE
      command: ["sh", "-c", "sleep infinity"]
      volumeMounts:
        - {name: data, mountPath: /recovery}
        - {name: config, mountPath: /etc/pgbackrest}
        - {name: secret, mountPath: /etc/pgbackrest-secret}
  volumes:
    - name: data
      persistentVolumeClaim: {claimName: $REC_PVC}
```

```

- name: config
  configMap: {name: pgbackrest-config}
- name: secret
  secret: {secretName: pgbackrest-repo-secret}
EOF
kubectl apply -f recovery-toolbox.yaml
kubectl -n "$NS" wait --for=condition=Ready \
  pod/pgbackrest-recovery-toolbox --timeout=10m

```

For a public training image, remove the imagePullSecrets stanza if no secret exists. If the pod fails to start, inspect its events. Confirm /recovery is the new PVC and empty before restore:

```

kubectl -n "$NS" describe pod pgbackrest-recovery-toolbox
kubectl -n "$NS" exec pgbackrest-recovery-toolbox -c toolbox -- \
  sh -c 'id; ls -la /recovery; test ! -e /recovery/pgroot/data/PG_VERSION'

```

Run this block only **inside the isolated recovery toolbox pod**, never inside acid-postgres-ha-0/1/2. Omitting --set selects the latest eligible backup. Verify the backup and WAL chain before startup.

```

kubectl -n "$NS" exec pgbackrest-recovery-toolbox -c toolbox -- \
  sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; \
  pgbackrest --stanza=postgres-ha info; \
  pgbackrest --stanza=postgres-ha \
  --pg1-path=/recovery/pgroot/data restore'
kubectl -n "$NS" exec pgbackrest-recovery-toolbox -c toolbox -- \
  sh -c 'test -f /recovery/pgroot/data/PG_VERSION && cat /recovery/pgroot/data/PG_VERSION'

```

8.3 Validate startup: The restored data must be started with PostgreSQL 18 and repository archive-get access. Spilo often generates configuration through its endpoint, so do not simply scale an operator-managed cluster onto the PVC. Use a separate, tested standalone recovery pod or an operator recovery mechanism supported by your installed version; confirm recovery log messages, query banking.dr_lab_events, and keep the recovery service separate from acid-postgres-ha. An offline PG_VERSION check proves files were restored, not that the database is recoverable.

For PITR, record a known-good UTC timestamp after a committed test row and before a simulated bad transaction in a disposable training database. Ensure WAL covering the target was archived. Reset **only** the recovery PVC, then run the following from its stopped toolbox container and start a separate recovery PostgreSQL instance to validate the good row and absence of the bad transaction.

9.1 Capture the point: First take a fresh full backup. Insert a known-good row. Read the target timestamp from the database clock, then wait a few seconds and delete the test row. This intentionally changes banking.dr_lab_events; do this only in your disposable lab environment.

```

kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc \
  'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; \
  pgbackrest --stanza=postgres-ha --type=full backup'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -d banking -c "INSERT INTO dr_lab_events(note)
  VALUES ('PITR good row');"
TARGET_TIME=$(kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -Atqc "SELECT to_char(clock_timestamp()
  AT TIME ZONE 'UTC', 'YYYY-MM-DD HH24:MI:SS.US') || '+00';")
echo "$TARGET_TIME"
sleep 3
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -d banking -c \
  "DELETE FROM dr_lab_events WHERE note='PITR good row';"
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -c 'SELECT pg_switch_wal();'

```

Use a new empty recovery PVC or stop the isolated recovery database and clear its data after confirming the PVC identity. If you choose a new PVC, update claimName in recovery-toolbox.yaml and recreate the toolbox. Verify the repository has WAL spanning TARGET_TIME before restore. Replace the placeholder in the command with the measured value; the example below passes that value into the pod explicitly.

```
kubectl -n "$NS" exec pgbackrest-recovery-toolbox -c toolbox -- \
  env TARGET_TIME="$TARGET_TIME" sh -lc \
  'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; \
  pgbackrest --stanza=postgres-ha \
  --pg1-path=/recovery/pgroot/data \
  --type=time --target="$TARGET_TIME" \
  --target-action=promote restore'
```

Replace the time and check the actual recovery PGDATA. No part of this step touches the live Patroni PVCs.

9. Troubleshooting and final validation

ImagePullBackOff: inspect pod events, private repository credentials, pod-template imagePullSecrets, and the tag. **Only pod 0 has pgBackRest:** compare the CR, StatefulSet, three pod imageIDs and rollout state; pooler pods are irrelevant. **Config missing:** inspect generated mounts. **Repository failure:** check GCS identity and bucket separately from Docker Hub. **Archive failure:** inspect pg_stat_archiver, effective Patroni config, and pgBackRest logs.

```
kubectl -n "$NS" get postgresql "$CLUSTER" \
  -o jsonpath='{.spec.dockerImage}{"\n\n"}'
kubectl -n "$NS" get statefulset "$CLUSTER" \
  -o jsonpath='{.spec.template.spec.containers[?(@.name=="postgres")].image}{"\n\n"}'
kubectl -n "$NS" describe pod "$CLUSTER-1"
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -x -c 'SELECT * FROM pg_stat_archiver;'
```

Completion: three database pods share the pgBackRest image; each reads configuration and secret; GCS stanza and WAL archive work; full/incremental/differential backups appear in info; isolated restore and PITR succeed. Record image digest, backup labels, RPO/RTO, and restore test result.

10. Failover, monitoring, retention, and cleanup

10.1 Failover: After Patroni elects a different primary, the original PRIMARY shell variable becomes stale. Repeat the role loop in section 0. Verify the new primary still has the custom image, pgBackRest config and cipher secret, and that its archive_command can reach GCS. A backup started during failover may fail and should be retried only after checking repository state.

```
for p in "$CLUSTER"-0 "$CLUSTER"-1 "$CLUSTER"-2; do
  kubectl -n "$NS" exec "$p" -c postgres -- \
    curl -fsS http://127.0.0.1:8008/ | jq -r '.role'
done
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- patronictl list
```

10.2 Daily monitoring: Record backup age, last archive success, failed archive count, repository capacity, and periodic isolated restore results. A successful backup job alone does not prove a tested recovery path. Retention must preserve the complete dependent backup sets and WAL required by the chosen recovery window.

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -x -c 'SELECT * FROM pg_stat_archiver;'
```

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -lc \
  'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; \
  pgbackrest --stanza=postgres-ha info'
```

10.3 Retention demonstration: repo1-retention-full=2 and repo1-retention-diff=4 are starting lab values. Review pgBackRest info and current command behavior before an expire operation, especially when an older backup might still be needed for a PITR target. Repository deletion is irreversible.

10.4 RPO and RTO: RPO is acceptable data loss measured from the incident to the last archived recoverable WAL. RTO is the elapsed time to provision storage, restore, replay WAL, validate, and cut over. A possible training schedule is full weekly, differential daily, incremental every 4-6 hours, continuous WAL archiving, and monthly restore drills; adjust to measured

RPO/RTO and data volume.

10.5 Cleanup: Keep your GCS repository and live PostgreSQL PVCs. Delete only disposable recovery objects after recording validation. Kubernetes PVC deletion may remove the backing storage depending on its reclaim policy, so inspect before deleting.

```
kubectl -n "$NS" delete pod pgbackrest-recovery-toolbox \
  --ignore-not-found
kubectl -n "$NS" get pvc "$REC_PVC"
# Delete the recovery PVC only after verifying its name and contents.
# kubectl -n "$NS" delete pvc "$REC_PVC"
```

Checklist: Logical dump verified; physical basebackup copied off pod; WAL archive verified; all three PostgreSQL members use the same custom image; private image pulls; mounted config and secret readable; GCS stanza and archive check pass; full/incremental/differential backups visible; isolated full restore and PITR validated; failover behavior observed; RPO/RTO recorded.

Reference documentation: Zalando Postgres Operator cluster manifest (dockerImage and additionalVolumes): https://github.com/zalando/postgres-operator/blob/master/docs/reference/cluster_manifest.md ; pgBackRest command reference (GCS, backup, archive, restore): <https://pgbackrest.org/command.html> . Verify these against the exact operator and pgBackRest versions running in your lab.